Programming ARM Boards

Unit II

Dr. Ayush Kumar Agrawal

SoCS, UPES

Introduction to ARM and STM32

Lecture Objectives

By the end of this lecture, students will be able to:

- Understand why ARM dominates modern embedded systems
- Differentiate between microcontrollers and SoCs
- Explain ARM licensing and Cortex families
- Identify STM32 architecture and ecosystem
- Understand how programming ARM boards differs from 8051 or Arduino

Why Study ARM-Based Systems?

- Modern embedded systems are no longer simple control units
- They require:
  - High computational capability
  - Real-time response
  - Low power consumption
  - Integration with communication protocols
- ARM processors provide the best balance between performance and power
- Over 90% of embedded processors today are ARM-based

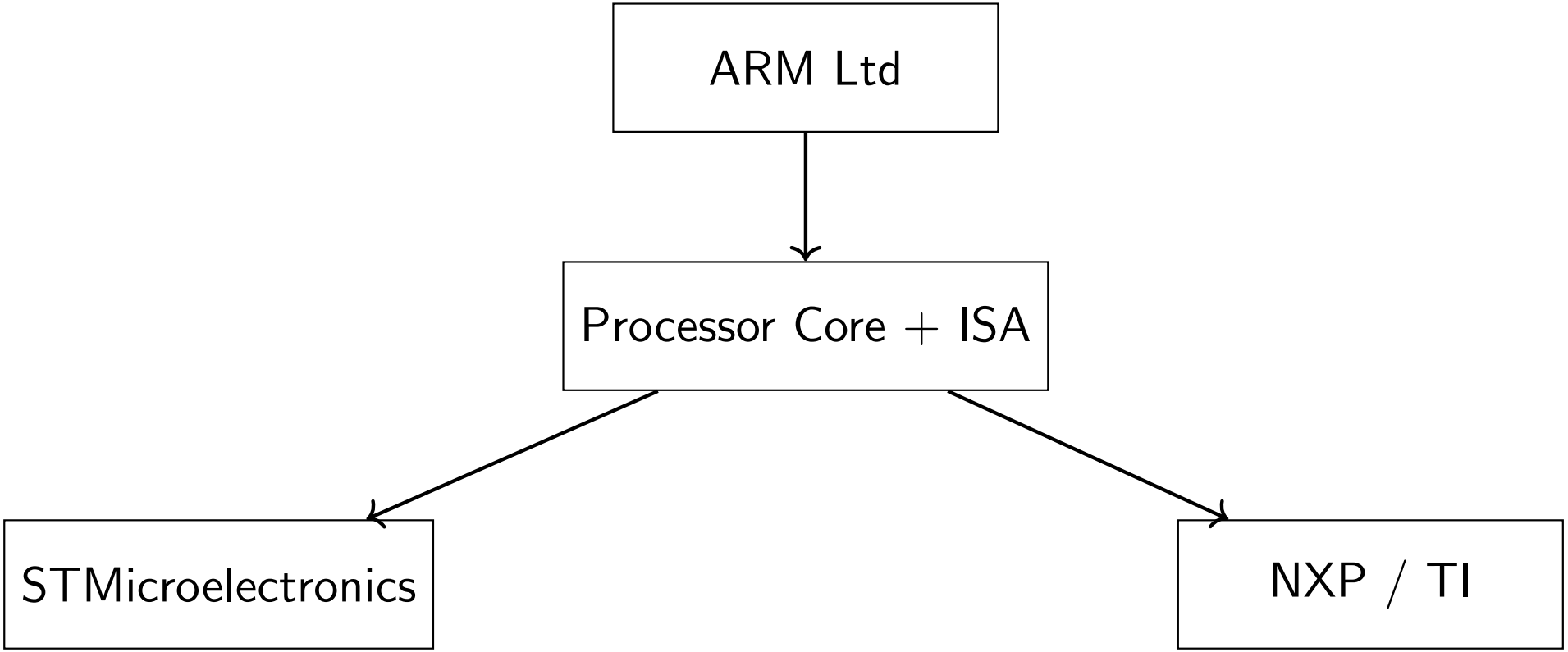
Evolution of Embedded Processors

- Early systems used:
  - 8-bit microcontrollers (8051, PIC)
  - Limited memory and peripherals
- Growth of IoT, robotics, and automation demanded:
  - 32-bit processing
  - Rich peripherals
  - Standardized architecture
- ARM emerged as the industry standard

What is ARM?

- ARM stands for Advanced RISC Machines
- It is not a chip manufacturer
- ARM designs:
  - Processor cores
  - Instruction Set Architecture (ISA)
- Semiconductor companies license ARM designs and build chips

ARM Licensing Model



ARM licenses designs; vendors integrate peripherals and memory

Microcontroller vs System-on-Chip

Microcontroller

A microcontroller integrates CPU, memory, and peripherals on a single chip for control-oriented tasks.

System-on-Chip

An SoC integrates processor, memory, buses, peripherals, and debug logic on a single silicon die.

- STM32 is a microcontroller-class SoC
- Everything required for execution is on-chip

ARM Cortex Processor Families

- Cortex-A:
  - Application processors
  - Runs Linux and Android
- Cortex-R:
  - Real-time processors
  - Automotive and safety-critical systems
- Cortex-M:
  - Microcontroller processors
  - Deterministic and low power

Why Cortex-M for Embedded Systems?

- Optimized for real-time applications
- Low interrupt latency
- Simple and efficient instruction set
- Integrated NVIC (Nested Vectored Interrupt Controller)
- Ideal for GPIO, timers, ADC, and communication protocols

Cortex-M Variants in STM32

- Cortex-M0/M0+: Ultra-low power, basic control
- Cortex-M3: Balanced performance, no FPU
- Cortex-M4: DSP instructions and FPU
- Cortex-M7: High-speed processing

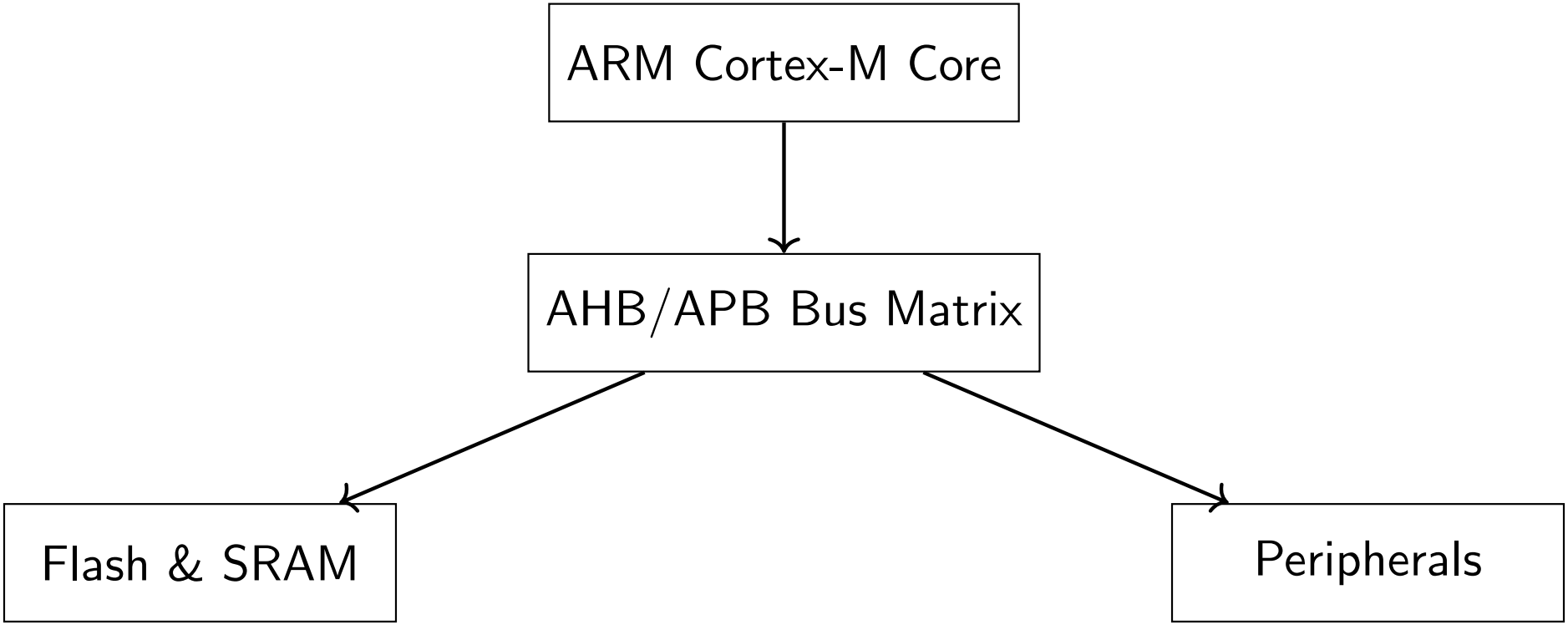
Course Focus

We mainly focus on Cortex-M3 and Cortex-M4 based STM32 boards

What is STM32?

- STM32 is a family of ARM Cortex-based microcontrollers
- Designed and manufactured by STMicroelectronics
- Widely used in:
  - Industrial automation
  - Robotics
  - IoT devices
  - Medical electronics

STM32 High-Level Architecture



STM32 Memory-Mapped Architecture

- All peripherals are mapped into memory address space
- GPIO, timers, ADC, UART accessed using addresses
- No separate I/O instructions
- Programming peripherals = reading/writing registers

Key Insight

Embedded programming is controlled manipulation of memory locations

STM32 Programming Philosophy

- Understand hardware through registers first
- Then move to abstraction layers
- Avoid black-box programming
- Enables:
  - Debugging
  - Optimization
  - Driver development

Programming Approaches in STM32

- Register-level programming
- CMSIS (Core support)
- HAL drivers
- Low Layer (LL) drivers

Course Strategy

Start from registers and gradually move upward

Development Tools Used

- STM32CubeIDE
- ARM GCC toolchain
- ST-Link debugger
- Datasheet and Reference Manual

What You Will Learn in Unit II

- GPIO driver development
- UART, SPI, and I2C communication
- ADC interfacing
- Timers and SysTick
- Interrupt and DMA programming

Lecture Summary

- ARM dominates modern embedded systems
- STM32 integrates ARM core with peripherals
- Everything is memory-mapped
- Strong fundamentals are essential before driver coding

GPIO Architecture and Programming

Lecture Objectives

After this lecture, students will be able to:

- Understand GPIO hardware architecture in STM32
- Explain GPIO pin modes and configurations
- Identify GPIO registers and their roles
- Program GPIO at register level
- Compare polling and interrupt-based GPIO

Why GPIO is Fundamental

- GPIO stands for General Purpose Input Output
- It is the simplest yet most critical peripheral
- Almost every embedded application starts with GPIO:
  - LED control
  - Button reading
  - Sensor interfacing
- Understanding GPIO builds confidence in hardware-level programming

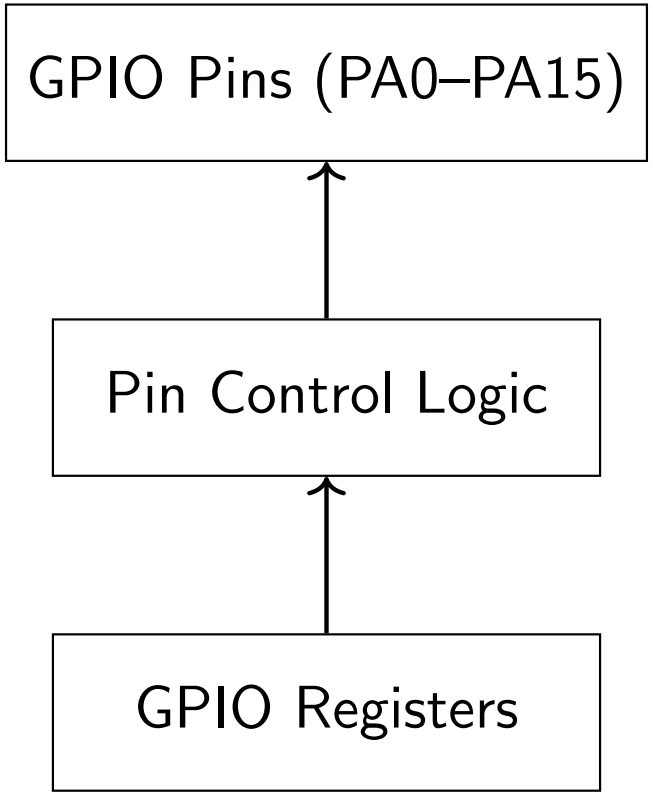
GPIO as an Interface Between Software and Hardware

- GPIO pins act as physical interfaces
- Software writes to registers
- Hardware reacts by changing voltage levels
- GPIO bridges digital logic and real-world signals

STM32 GPIO Port Organization

- GPIO pins are grouped into ports
- Each port is named alphabetically: GPIOA, GPIOB, GPIOC, etc.
- Each port typically has 16 pins:
  - PA0 to PA15
  - PB0 to PB15
- Each port has a dedicated set of registers

GPIO Port Structure



Registers control the behavior of physical pins

GPIO Pin Modes

Each GPIO pin can be configured in one of the following modes:

- Input mode
- Output mode
- Alternate function mode
- Analog mode

Important

Mode selection decides how the pin interacts with internal circuitry

Input Mode

- Pin reads external signal
- Used for buttons, switches, sensors
- Input can be:
  - Floating
  - Pull-up
  - Pull-down
- Pull resistors prevent undefined logic levels

Output Mode

- Pin drives external load
- Used for LEDs, relays, control signals
- Output types:
  - Push-pull
  - Open-drain
- Output speed controls rise and fall time

Alternate Function Mode

- Pin controlled by peripheral instead of GPIO logic
- Used for:
  - UART
  - SPI
  - I2C
  - Timers
- Multiplexing allows one pin to serve multiple functions

Analog Mode

- Disconnects digital input/output circuitry
- Used for ADC and DAC
- Reduces power consumption
- Improves analog accuracy

Key GPIO Registers Overview

Each GPIO port has the following major registers:

- MODER – Mode register
- OTYPER – Output type register
- OSPEEDR – Output speed register
- PUPDR – Pull-up pull-down register
- IDR – Input data register
- ODR – Output data register
- BSRR – Bit set/reset register

GPIO MODER Register

- Controls pin mode
- Each pin uses 2 bits
- Configuration:
  - 00 – Input
  - 01 – Output
  - 10 – Alternate function
  - 11 – Analog

Exam Tip

MODER is always the first register to configure

GPIO Output Registers

- ODR reflects output state
- Writing 1 drives pin high
- Writing 0 drives pin low
- BSRR allows atomic bit set/reset

Why BSRR?

Prevents race conditions during read-modify-write

GPIO Input Register

- IDR holds current pin logic level
- Read-only register
- Used for polling-based input reading

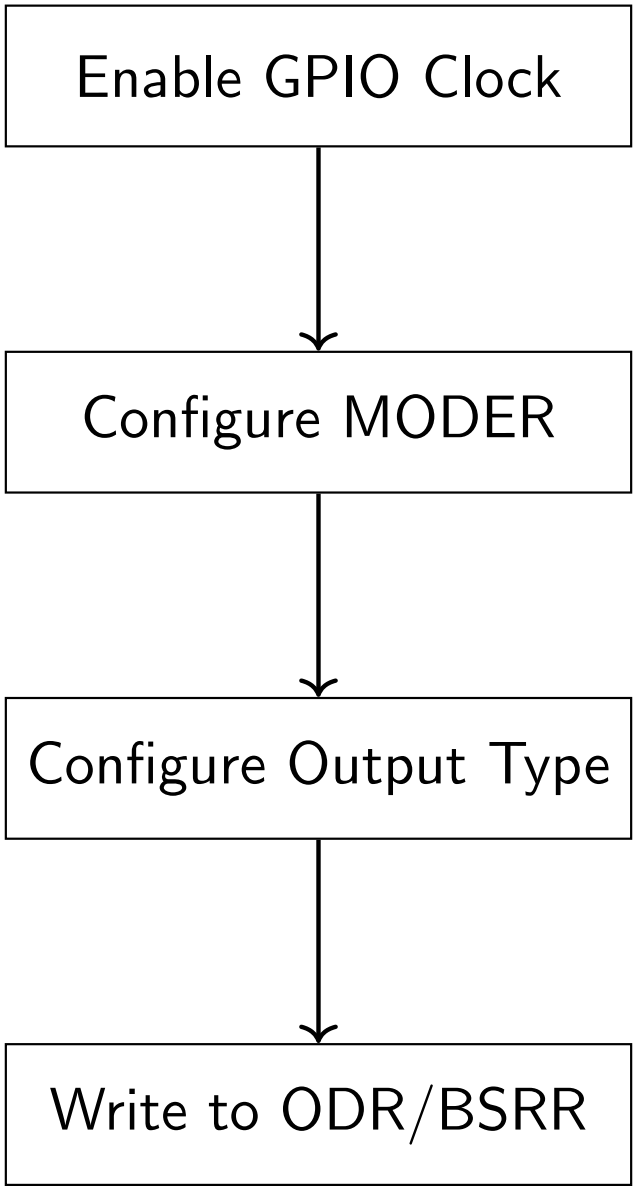
GPIO Register Access Concept

- Registers are memory-mapped
- Accessed using pointers in C
- Writing to address modifies hardware

Core Concept

Software does not touch pins directly, it touches registers

Basic GPIO Output Flow



GPIO Polling vs Interrupt

- Polling:
  - CPU continuously checks pin
  - Simple but inefficient
- Interrupt:
  - Hardware notifies CPU
  - Efficient and scalable

Note

Interrupt-based GPIO will be covered in a later lecture

Common GPIO Programming Mistakes

- Forgetting to enable GPIO clock
- Incorrect MODER bit calculation
- Using ODR instead of BSRR
- Floating input pins

GPIO in Real Applications

- LED blinking
- Button debouncing
- Sensor trigger signals
- Control of external modules

Lecture Summary

- GPIO is the foundation of embedded systems
- STM32 GPIO is register-controlled
- MODER defines pin behavior
- Correct sequencing is essential

GPIO Programming: Register-Level Coding and Examples

Lecture Objectives

After this lecture, students will be able to:

- Write register-level GPIO programs in C
- Enable GPIO peripheral clocks
- Configure GPIO pins as input and output
- Control LEDs using GPIO
- Read button input using polling
- Understand bit manipulation in embedded C

Prerequisite Knowledge

Before coding GPIO, students must understand:

- Memory-mapped I/O
- GPIO register set
- Bitwise operators in C
- STM32 reference manual structure

Register-Level Programming Philosophy

- No HAL (Hardware Abstruction Layer)
- No library abstraction
- Direct interaction with hardware registers
- Full control and transparency

Key Advantage

You know exactly what the hardware is doing

Step 1: Understanding Base Addresses

- Peripheral registers have fixed base addresses
- Example for STM32:
  - Reset and Clock Control (RCC) base address
  - GPIOA base address
- Offsets are added to base address to access registers

Important GPIO Registers Used

- RCC AHB1ENR
- GPIOx MODER
- GPIOx OTYPER
- GPIOx OSPEEDR
- GPIOx PUPDR
- GPIOx ODR
- GPIOx IDR
- GPIOx BSRR

Step 2: Enabling GPIO Clock

- GPIO will not work unless its clock is enabled
- RCC controls clock distribution
- Each GPIO port has a corresponding enable bit

Concept

No clock means no register access

Clock Enable Code Example

```
#define RCC_AHB1ENR  (*(volatile unsigned int*)0x40023830)

void gpio_clock_enable(void)
{
 RCC_AHB1ENR |= (1 << 0);   // Enable GPIOA clock
}
```

Why volatile Keyword is Mandatory

- Prevents compiler optimization
- Register value can change due to hardware
- Ensures actual memory access

Exam Point

Missing volatile causes unpredictable behavior

Step 3: GPIO Pin Mode Configuration

- MODER register controls pin mode
- Each pin uses 2 bits
- Example:
  - 00 input
  - 01 output

Configuring PA5 as Output

```
#define GPIOA_MODER (*(volatile unsigned int*)0x40020000)

void gpio_output_config(void)
{
 GPIOA_MODER &= ~(3 << (5 * 2));
 GPIOA_MODER |=  (1 << (5 * 2));
}
```

Explanation of MODER Code

- Clear both bits for pin 5
- Set bit pattern to 01
- Pin becomes general-purpose output

Step 4: Writing Output Data

- Use ODR or BSRR
- BSRR is preferred
- Atomic operation

LED ON and OFF Using BSRR

```
#define GPIOA_BSRR (*(volatile unsigned int*)0x40020018)

void led_on(void)
{
 GPIOA_BSRR = (1 << 5);
}

void led_off(void)
{
 GPIOA_BSRR = (1 << (5 + 16));
}
```

Why Use BSRR Instead of ODR

- Avoids read-modify-write issue
- Safe in interrupt-driven systems
- Faster execution

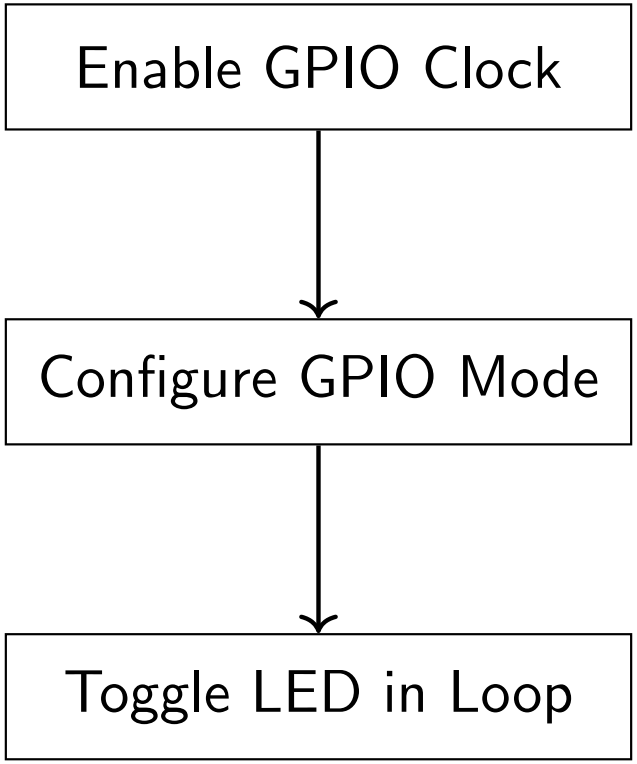
Step 5: Creating Delay

- Simple software delay loop
- Not accurate but useful for learning

Simple Delay Function

```
void delay(volatile unsigned int count)
{
 while(count--);
}
```

Complete LED Blinking Program Flow



Step 6: GPIO Input Reading

- Configure pin as input
- Read IDR register
- Polling-based input

Button Input Example

```
#define GPIOA_IDR (*(volatile unsigned int*)0x40020010)

int read_button(void)
{
 return (GPIOA_IDR & (1 << 0)) ? 1 : 0;
}
```

Polling Logic Explanation

- CPU continuously checks input
- Simple but inefficient
- Suitable for basic applications

Common GPIO Coding Errors

- Clock not enabled
- Wrong bit shift calculation
- Missing volatile
- Floating input pins

Exam-Oriented Questions

- Why is BSRR preferred over ODR?
- Explain role of MODER register
- What happens if GPIO clock is not enabled?
- Explain use of volatile keyword

Lecture Summary

- GPIO can be fully controlled using registers
- Clock enable is mandatory
- Bit manipulation is the heart of embedded C
- This approach builds strong debugging skills

USART Protocol and Serial Communication Basics

Lecture Objectives

After this lecture, students will be able to:

- Understand serial communication fundamentals
- Differentiate between synchronous and asynchronous communication
- Explain UART vs USART
- Understand USART frame structure
- Interpret baud rate concept
- Identify USART signals in STM32

Why Serial Communication is Needed

- Embedded systems rarely work in isolation
- Communication required for:
  - Debugging via PC
  - Sensor data exchange
  - Module interfacing (GPS, GSM, Bluetooth)
- Serial communication reduces wiring complexity

Parallel vs Serial Communication

Parallel

- Multiple data lines
- Faster over short distance
- Complex wiring

Serial

- Single data line
- Slower but efficient
- Long distance support

Serial Communication Types

- Synchronous communication
- Asynchronous communication

Difference

Synchronous uses clock signal Asynchronous does not use shared clock

Asynchronous Communication Concept

- No clock line
- Data synchronized using start and stop bits
- Sender and receiver must agree on baud rate

UART vs USART

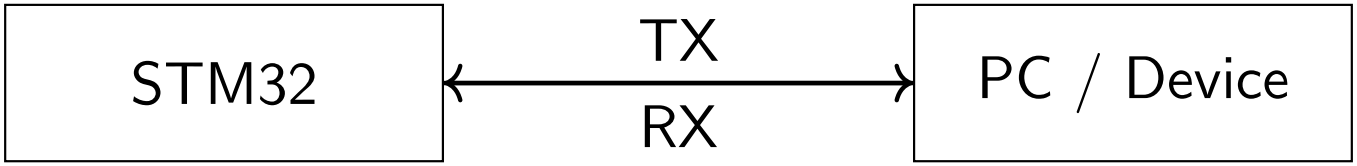
- UART: Universal Asynchronous Receiver Transmitter
- USART: Universal Synchronous Asynchronous Receiver Transmitter
- USART supports:
  - Asynchronous mode
  - Synchronous mode

In STM32

USART is mostly used in asynchronous mode

USART Basic Signals

- TX – Transmit
- RX – Receive
- GND – Common ground



USART Frame Structure

- Start bit
- Data bits (5 to 9 bits)
- Optional parity bit
- Stop bit(s)

USART Frame Format Diagram

Start	Data Bits	Parity	Stop

Idle line is HIGH. Transmission begins with LOW start bit.

Baud Rate Concept

- Baud rate = number of symbols per second
- Common values:
  - 9600
  - 115200
- Sender and receiver must match baud rate

Example

At 9600 baud, 9600 bits transmitted per second

Data Framing Example

- Suppose transmitting ASCII 'A'
- ASCII of 'A' = 65 = 01000001
- Frame becomes:
  - Start bit = 0
  - Data bits = 01000001
  - Stop bit = 1

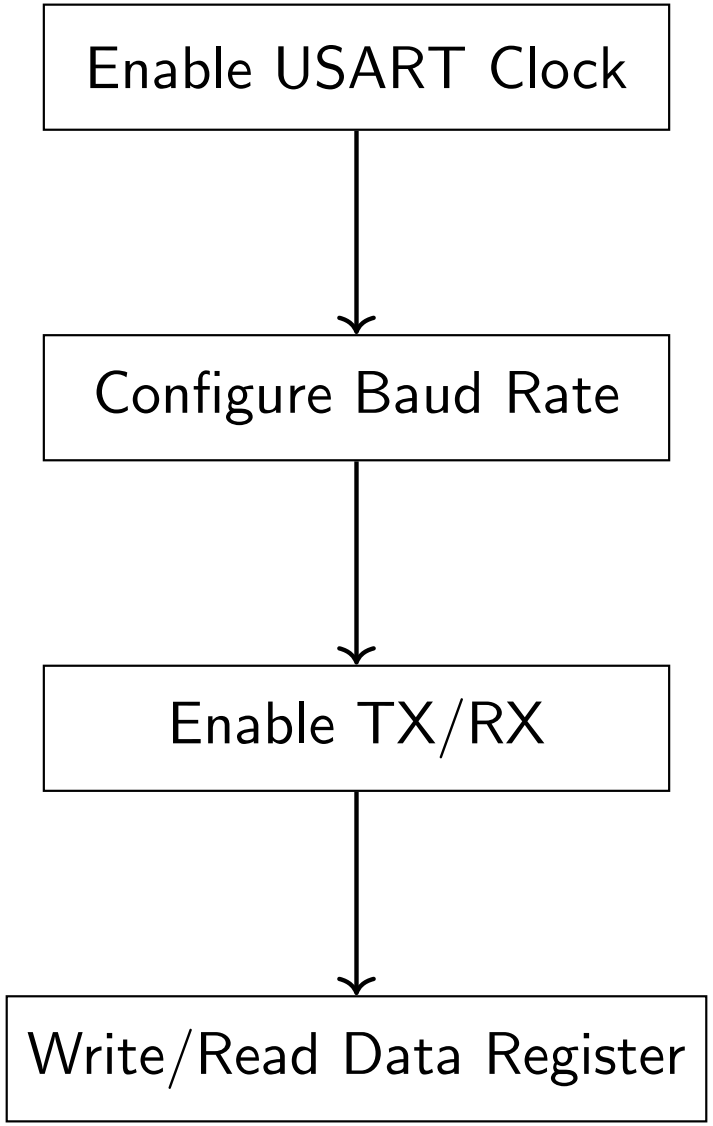
Parity Bit

- Used for simple error detection
- Even parity
- Odd parity
- Optional in most STM32 applications

USART in STM32 Architecture

- USART is connected to APB bus
- Has dedicated registers:
  - SR – Status register
  - DR – Data register
  - BRR – Baud rate register
  - CR1, CR2, CR3 – Control registers

USART Communication Flow



Common USART Applications

- Serial debugging
- GPS module interfacing
- GSM communication
- Bluetooth module
- Industrial control systems

Common Errors in USART Communication

- Baud rate mismatch
- No common ground
- Incorrect TX/RX connection
- Wrong frame format

Exam-Oriented Questions

- Differentiate UART and USART
- Explain USART frame format with diagram
- Define baud rate
- What happens if baud rates mismatch?

Lecture Summary

- USART enables serial communication
- Operates in synchronous and asynchronous mode
- Frame consists of start, data, parity, stop bits
- Correct configuration is mandatory for communication

USART Register-Level Programming in STM32

Lecture Objectives

After this lecture, students will be able to:

- Configure USART at register level
- Calculate baud rate value
- Transmit a character using polling
- Receive a character using polling
- Develop basic serial communication driver

Step 1: USART Initialization Overview

To initialize USART, the following steps are required:

1. Enable USART clock
2. Configure GPIO pins in Alternate Function mode
3. Configure baud rate
4. Enable transmitter and receiver
5. Enable USART module

Important USART Registers

- SR - Status Register
- DR - Data Register
- BRR - Baud Rate Register
- CR1 - Control Register 1
- CR2 - Control Register 2
- CR3 - Control Register 3

Step 2: Enabling USART Clock

USART1 is connected to APB2 bus.

Clock Enable

Bit in RCC APB2ENR must be set.

USART Clock Enable Code

```
#define RCC_APB2ENR (*(volatile unsigned int*)0x40023844)

void usart1_clock_enable(void)
{
 RCC_APB2ENR |= (1 << 4);  // Enable USART1 clock
}
```

Step 3: GPIO Configuration for USART

For USART1:

- PA9 - TX
- PA10 - RX

Pins must be configured in Alternate Function mode.

GPIO Alternate Function Setup

```
#define GPIOA_MODER (*(volatile unsigned int*)0x40020000)
#define GPIOA_AFRH  (*(volatile unsigned int*)0x40020024)

void usart_gpio_config(void)
{
 // Set PA9 and PA10 to Alternate Function mode
 GPIOA_MODER &= ~(0xF << (9 * 2));
 GPIOA_MODER |=  (0xA << (9 * 2));

 // Select AF7 for USART1
 GPIOA_AFRH &= ~(0xFF << 4);
 GPIOA_AFRH |=  (0x77 << 4);
}
```

Step 4: Baud Rate Calculation

Baud Rate Formula:

$$USARTDIV = \frac{f_{CK}}{16 \times BaudRate}$$

Example:

$$f_{CK} = 16MHz, \quad BaudRate = 9600$$

$$USARTDIV = 104.16$$

This value is programmed into BRR register.

Setting Baud Rate

```
#define USART1_BRR (*(volatile unsigned int*)0x40011008)

void usart_set_baud(void)
{
 USART1_BRR = 0x683;  // Approx value for 9600 @16MHz
}
```

Step 5: Enabling USART

CR1 register controls USART enable, TX enable, RX enable.

USART Enable Code

```
#define USART1_CR1 (*(volatile unsigned int*)0x4001100C)

void usart_enable(void)
{
 USART1_CR1 |= (1 << 13);  // UE bit
 USART1_CR1 |= (1 << 3);   // TE bit
 USART1_CR1 |= (1 << 2);   // RE bit
}
```

Transmitting Data (Polling Method)

- Wait until TXE flag is set
- Write data into DR register

USART Transmit Function

```
#define USART1_SR (*(volatile unsigned int*)0x40011000)
#define USART1_DR (*(volatile unsigned int*)0x40011004)

void usart1_tx(char data)
{
 while(!(USART1_SR & (1 << 7)));  // Wait for TXE
 USART1_DR = data;
}
```

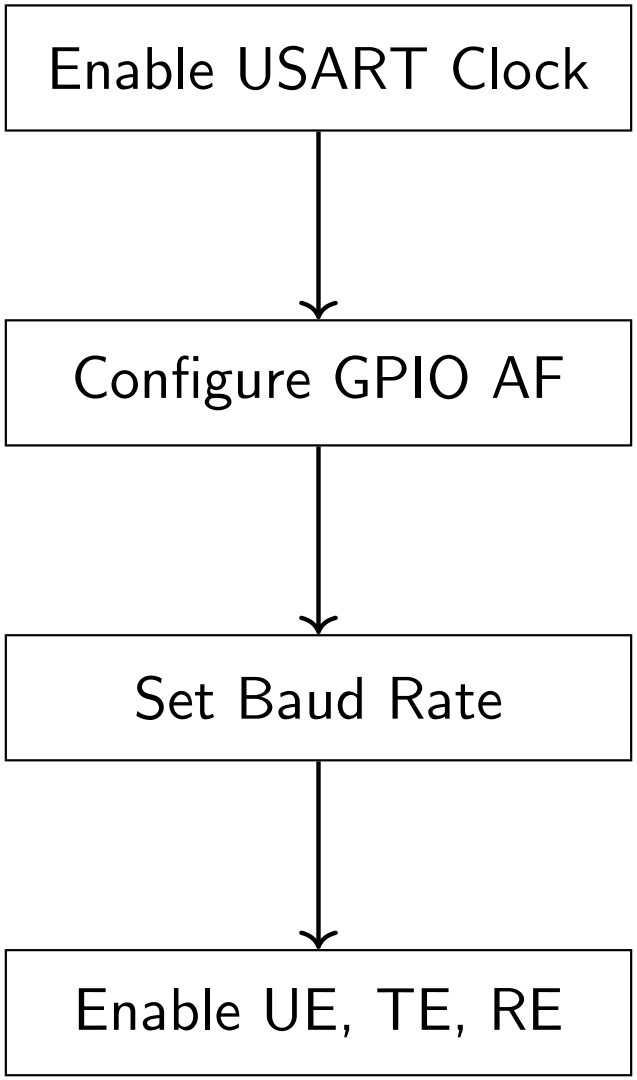
Receiving Data (Polling Method)

- Wait until RXNE flag is set
- Read DR register

USART Receive Function

```
char usart1_rx(void)
{
 while(!(USART1_SR & (1 << 5)));  // Wait for RXNE
 return USART1_DR;
}
```

Complete USART Initialization Flow



Exam-Oriented Questions

- Derive baud rate formula
- Explain role of BRR register
- What are TXE and RXNE flags?
- Why polling is inefficient?

Lecture Summary

- USART can be fully controlled using registers
- Clock, GPIO AF, Baud rate, CR1 are mandatory steps
- TXE and RXNE flags manage data flow
- Polling method is simple but CPU intensive

USART Interrupt-Based Communication

Lecture Objectives

After this lecture, students will be able to:

- Understand interrupt-driven communication
- Explain NVIC architecture in Cortex-M
- Configure USART interrupt
- Write ISR for RX interrupt
- Compare polling vs interrupt methods

Why Move Beyond Polling?

- Polling wastes CPU cycles
- CPU continuously checks RXNE flag
- Inefficient in multitasking systems
- Interrupt allows CPU to perform other tasks

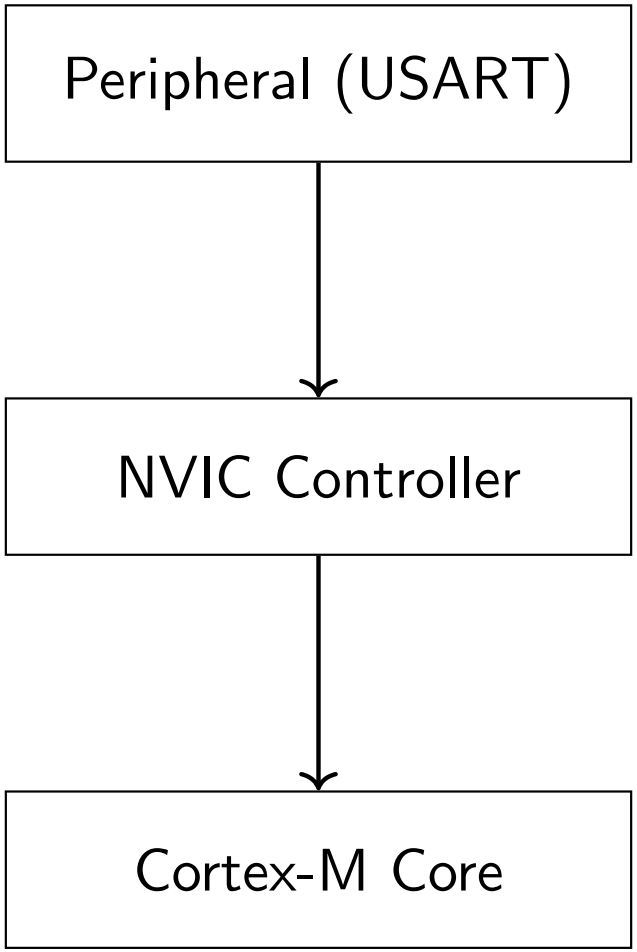
Core Idea

Interrupts make embedded systems efficient and scalable

Interrupt Concept

- Peripheral generates signal
- CPU suspends current task
- Executes Interrupt Service Routine (ISR)
- Returns to previous execution

Cortex-M NVIC Architecture



NVIC Features

- Nested interrupts
- Priority levels
- Fast context switching
- Hardware vector table

USART Interrupt Flow

1. Data received in USART
2. RXNE flag set
3. Interrupt request sent to NVIC
4. NVIC triggers ISR
5. ISR reads data register

Registers Required for Interrupt

- USART CR1 (RXNEIE bit)
- NVIC ISER register
- USART SR register

Enable USART RX Interrupt

```
#define USART1_CR1 (*(volatile unsigned int*)0x4001100C)

void usart1_enable_interrupt(void)
{
 USART1_CR1 |= (1 << 5);  // RXNEIE bit
}
```

Enabling NVIC for USART1

USART1 interrupt number for many STM32 variants is 37. NVIC ISER register enables interrupt.

Enable NVIC Interrupt

```
#define NVIC_ISER1 (*(volatile unsigned int*)0xE000E104)

void nvic_enable_usart1(void)
{
 NVIC_ISER1 |= (1 << 5);  // Enable IRQ37
}
```

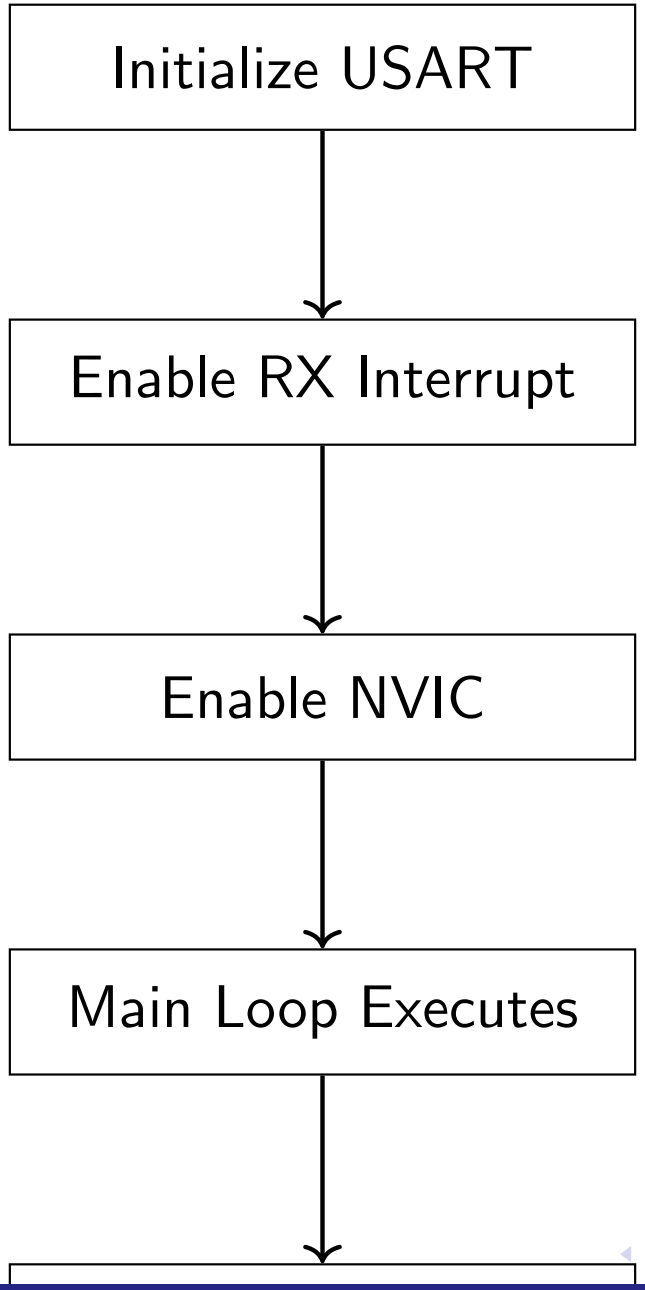
Writing the ISR

- ISR name must match vector table
- Automatically executed on interrupt
- Must be fast and efficient

USART1 Interrupt Handler

```
#define USART1_SR (*(volatile unsigned int*)0x40011000)
#define USART1_DR (*(volatile unsigned int*)0x40011004)

volatile char received_data;

void USART1_IRQHandler(void)
{
 if(USART1_SR & (1 << 5))   // RXNE flag
 {
  received_data = USART1_DR;
 }
}
```

Complete Interrupt-Based Flow



Polling vs Interrupt Comparison

Feature	Polling	Interrupt
CPU Usage	High	Low
Efficiency	Low	High
Complexity	Simple	Moderate
Scalability	Poor	Excellent

Common Mistakes

- Forgetting to enable NVIC
- Not clearing interrupt flags
- Long ISR execution time
- Wrong ISR function name

Exam-Oriented Questions

- Explain NVIC architecture
- Differentiate polling and interrupt
- Explain RXNE interrupt process
- Why ISR should be short?

Lecture Summary

- Interrupts improve efficiency
- NVIC manages interrupt prioritization
- USART interrupt uses RXNE flag
- ISR must be optimized and precise

System Tick (SysTick) Timer and General Purpose Timers

Lecture Objectives

After this lecture, students will be able to:

- Understand the need for hardware timers
- Explain SysTick architecture in Cortex-M
- Configure SysTick for precise delay
- Implement millisecond delay using SysTick
- Understand interrupt-based timing

Why Software Delay is Not Recommended

- Depends on compiler optimization
- Depends on clock frequency
- Blocks CPU execution
- Not scalable

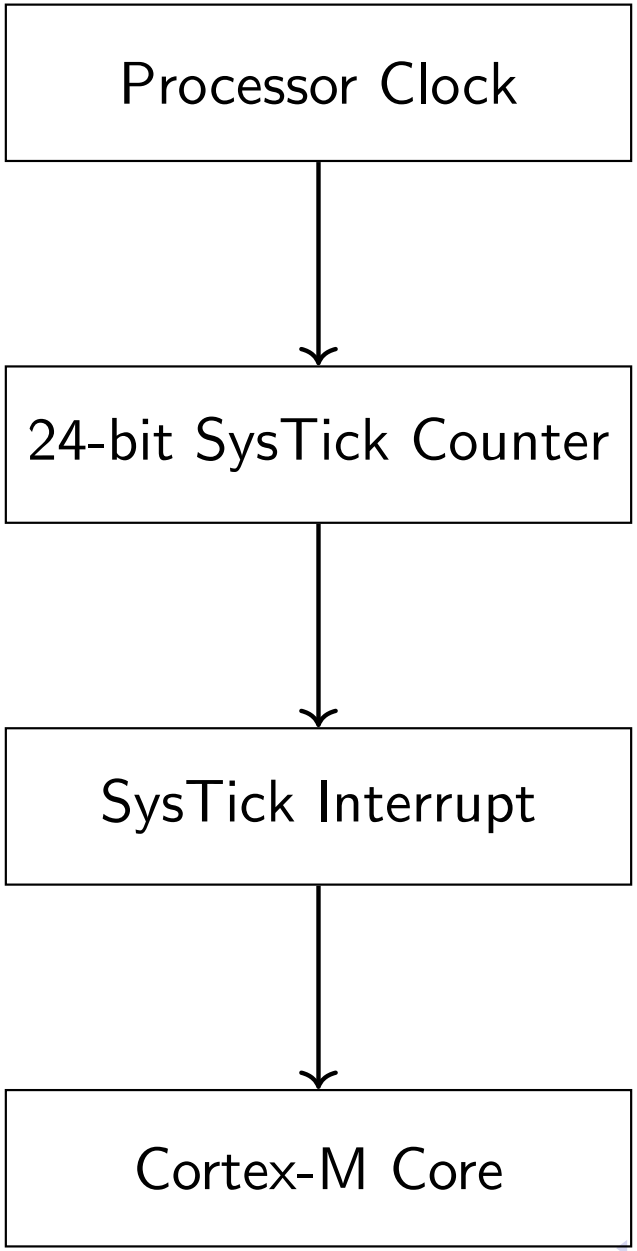
Conclusion

Accurate embedded systems must use hardware timers

What is SysTick?

- SysTick is a 24-bit down counter
- Integrated inside Cortex-M core
- Used for:
  - System delays
  - OS time base
  - Periodic interrupts
- Does not belong to APB or AHB bus

SysTick Architecture



SysTick Registers

- CTRL - Control and Status Register
- LOAD - Reload Value Register
- VAL - Current Value Register
- CALIB - Calibration Register

SysTick CTRL Register Bits

- Bit 0 - ENABLE
- Bit 1 - TICKINT (Interrupt enable)
- Bit 2 - CLKSOURCE
- Bit 16 - COUNTFLAG

Important

COUNTFLAG becomes 1 when counter reaches zero

Delay Calculation Formula

If system clock = 16 MHz
For 1 ms delay:

$$Reload = \frac{Clock}{1000} - 1$$

$$Reload = \frac{16000000}{1000} - 1 = 15999$$

SysTick Delay Initialization

```
#define SYST_CSR  (*(volatile unsigned int*)0xE000E010)
#define SYST_RVR  (*(volatile unsigned int*)0xE000E014)
#define SYST_CVR  (*(volatile unsigned int*)0xE000E018)

void systick_init(void)
{
 SYST_RVR = 15999;      // 1 ms delay
 SYST_CVR = 0;          // Clear current value
 SYST_CSR = 0x5;        // Enable SysTick, use processor clo
}
```

Blocking Delay Using COUNTFLAG

- Wait until COUNTFLAG becomes 1
- Indicates counter reached zero

Millisecond Delay Function

```
void delay_ms(int ms)
{
 for(int i=0; i<ms; i++)
 {
  while(!(SYST_CSR & (1 << 16)));
 }
}
```

Interrupt-Based SysTick

- Enable TICKINT bit
- SysTick interrupt generated periodically
- ISR increments global time variable

SysTick Interrupt Handler

```
volatile unsigned int tick_count = 0;

void SysTick_Handler(void)
{
 tick_count++;
}
```

Using SysTick as System Clock

- $tick_count$ stores milliseconds
- Used in:
  - RTOS scheduling
  - Timeout implementation
  - Task delays

Polling vs Interrupt SysTick

Feature	Polling	Interrupt
CPU Blocking	Yes	No
Efficiency	Low	High
Use in RTOS	No	Yes
Precision	Good	Excellent

Common Mistakes

- Incorrect reload calculation
- Forgetting to clear current value
- Not enabling clock source
- Heavy processing inside SysTick ISR

Exam-Oriented Questions

- Explain SysTick architecture
- Derive delay calculation formula
- Differentiate blocking vs interrupt delay
- Why SysTick is preferred in RTOS?

Lecture Summary

- SysTick is a core-level timer
- Provides accurate millisecond timing
- Supports blocking and interrupt-based delays
- Foundation for real-time systems

General Purpose Timers in STM32

Lecture Objectives

After this lecture, students will be able to:

- Understand STM32 timer architecture
- Configure prescaler and auto-reload register
- Generate time delay using TIMx
- Generate PWM signals
- Understand input capture and output compare

Why General Purpose Timers?

- SysTick is limited (24-bit)
- Industrial systems require:
  - Microsecond resolution
  - PWM generation
  - Frequency measurement
  - Pulse width measurement
- TIMx modules provide flexible timing

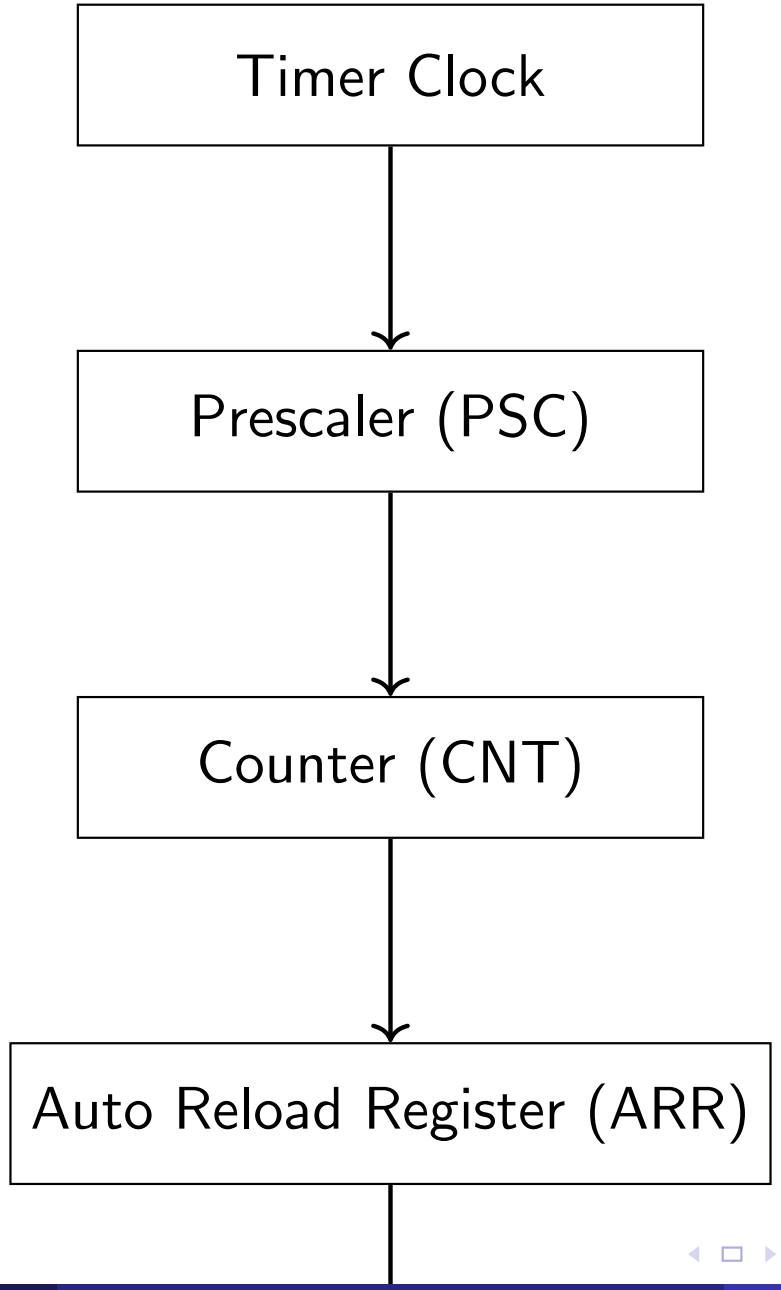
STM32 Timer Types

- Basic Timers (TIM6, TIM7)
- General Purpose Timers (TIM2–TIM5)
- Advanced Control Timers (TIM1, TIM8)

Focus

This lecture focuses on General Purpose Timers

Timer Architecture Overview



Prescaler Concept

- Timer clock often high frequency
- Prescaler divides clock

$$TimerClock = \frac{SystemClock}{PSC + 1}$$

Example

SystemClock = 16 MHz PSC = 15999 TimerClock = 1 kHz

Auto Reload Register (ARR)

- Defines maximum counter value
- Counter counts from 0 to ARR
- When counter equals ARR:
  - Update event generated
  - Counter resets

Timer Delay Calculation

To generate 1 second delay:
Step 1: Choose PSC for 1 kHz timer clock Step 2: Set ARR = 1000

$$Delay = \frac{(PSC + 1)(ARR + 1)}{SystemClock}$$

Important Timer Registers

- TIMx_PSC – Prescaler
- TIMx_ARR – Auto reload
- TIMx_CNT – Counter
- TIMx_CR1 – Control register
- TIMx_DIER – Interrupt enable
- TIMx_SR – Status register

Timer Initialization Example (TIM2)

```
#define RCC_APB1ENR (*(volatile unsigned int*)0x40023840)
#define TIM2_PSC    (*(volatile unsigned int*)0x40000028)
#define TIM2_ARR    (*(volatile unsigned int*)0x4000002C)
#define TIM2_CR1    (*(volatile unsigned int*)0x40000000)

void tim2_init(void)
{
 RCC_APB1ENR |= (1 << 0);  // Enable TIM2 clock

 TIM2_PSC = 15999;         // 1 kHz clock
 TIM2_ARR = 1000;          // 1 second
 TIM2_CR1 |= (1 << 0);     // Enable counter
}
```

PWM Concept

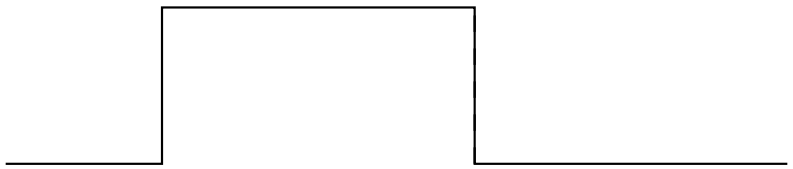
- Pulse Width Modulation controls power
- Used in:
  - Motor control
  - LED brightness
  - Servo control

PWM Duty Cycle

$$DutyCycle = \frac{CCR}{ARR} \times 100$$

If ARR = 1000 CCR = 500
Duty Cycle = 50%

PWM Signal Representation



High time determined by CCR register

Input Capture

- Measures external signal frequency
- Captures timer value on edge detection
- Used in:
  - Frequency measurement
  - Pulse width detection

Output Compare

- Generates output signal when counter matches CCR
- Used for waveform generation

Timer Interrupt

- Enable update interrupt in DIER
- Check UIF flag in SR
- Clear flag inside ISR

Polling vs Interrupt Timer

Feature	Polling	Interrupt
CPU Blocking	Yes	No
Precision	Moderate	High
Scalability	Low	High
Industrial Use	Rare	Common

Common Mistakes

- Wrong prescaler calculation
- Forgetting to enable clock
- Not clearing update flag
- Incorrect PWM mode selection

Exam-Oriented Questions

- Derive timer delay formula
- Explain PSC and ARR role
- Explain PWM generation using timer
- Differentiate input capture and output compare

Lecture Summary

- TIMx provides flexible timing control
- Prescaler divides clock
- ARR defines period
- CCR defines duty cycle
- Essential for real-world embedded applications

ADC Driver Development in STM32

Lecture Objectives

After this lecture, students will be able to:

- Understand ADC architecture in STM32
- Configure ADC in register-level programming
- Perform single channel conversion
- Read analog data using polling
- Understand interrupt-based ADC preview

Why ADC is Important?

- Real-world signals are analog
- Microcontrollers process digital data
- ADC converts analog voltage to digital value

Applications

Temperature sensors, LDR, Gas sensors, Biomedical signals

ADC Conversion Concept

$$DigitalValue = \frac{InputVoltage}{V_{ref}} \times (2^n - 1)$$

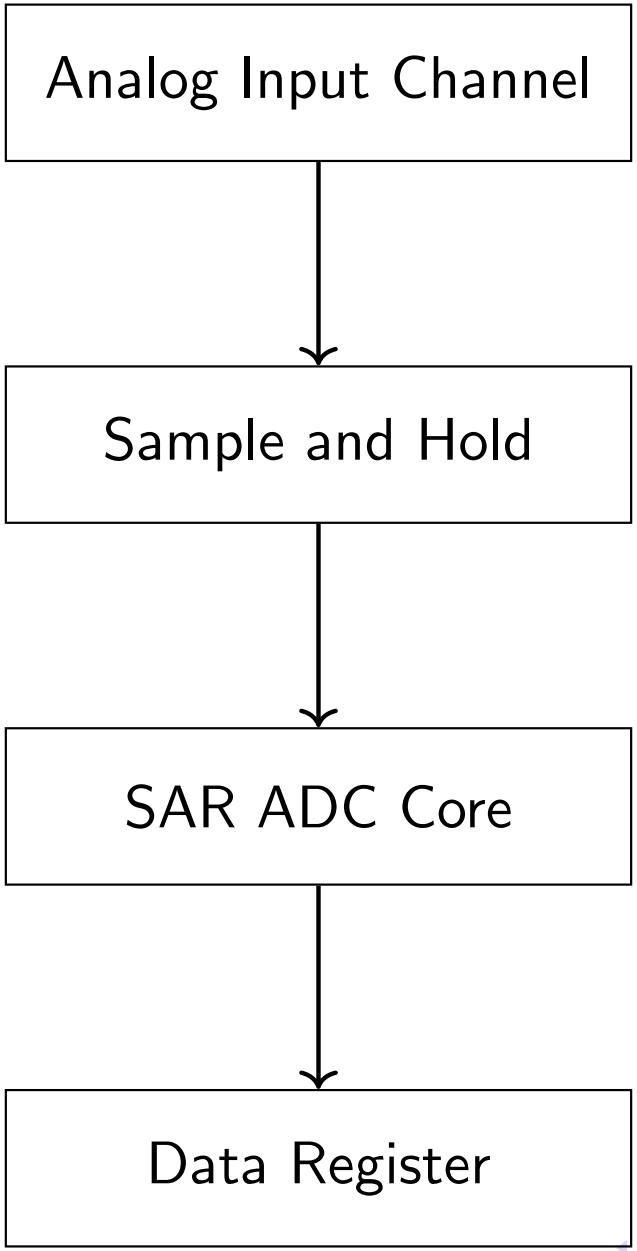
For 12-bit ADC:

$$Resolution = \frac{V_{ref}}{4096}$$

If Vref = 3.3V:

$$Resolution \approx 0.8mV$$

STM32 ADC Architecture



Important ADC Registers

- ADC_CR1 – Control Register 1
- ADC_CR2 – Control Register 2
- ADC_SMPR – Sampling Time Register
- ADC_SQR – Sequence Register
- ADC_DR – Data Register

ADC Initialization Steps

1. Enable ADC clock
2. Configure GPIO pin as analog
3. Select ADC channel
4. Configure sampling time
5. Enable ADC
6. Start conversion

Enable ADC Clock

```
#define RCC_APB2ENR (*(volatile unsigned int*)0x40023844)

void adc_clock_enable(void)
{
 RCC_APB2ENR |= (1 << 8);  // Enable ADC1
}
```

Configure GPIO as Analog

- Set MODER bits to 11
- Disable pull-up/pull-down

GPIO Analog Mode Example

```
#define GPIOA_MODER (*(volatile unsigned int*)0x40020000)

void gpio_analog_config(void)
{
 GPIOA_MODER |= (3 << (1 * 2));  // PA1 analog mode
}
```

Selecting ADC Channel

- SQR3 register selects first conversion
- Example: Channel 1

ADC Channel Selection

```
#define ADC1_SQR3 (*(volatile unsigned int*)0x40012034)

void adc_channel_select(void)
{
 ADC1_SQR3 = 1;   // Channel 1
}
```

Enable ADC and Start Conversion

- Set ADON bit
- Set SWSTART bit

ADC Start Conversion

```
#define ADC1_CR2 (*(volatile unsigned int*)0x40012008)

void adc_start(void)
{
 ADC1_CR2 |= (1 << 0);   // ADON
 ADC1_CR2 |= (1 << 30);  // SWSTART
}
```

Reading ADC Value (Polling)

- Wait until EOC flag set
- Read ADC_DR register

ADC Read Function

```
#define ADC1_SR (*(volatile unsigned int*)0x40012000)
#define ADC1_DR (*(volatile unsigned int*)0x4001204C)

unsigned int adc_read(void)
{
 while(!(ADC1_SR & (1 << 1)));  // Wait for EOC
 return ADC1_DR;
}
```

ADC Interrupt Preview

- Enable EOCIE bit
- NVIC configuration required
- ISR reads data register

Polling vs Interrupt ADC

Feature	Polling	Interrupt
CPU Usage	High	Low
Efficiency	Moderate	High
Complexity	Simple	Medium
Industrial Use	Rare	Common

Common Mistakes

- Forgetting analog mode
- Not enabling ADC clock
- Wrong channel selection
- Not clearing EOC flag

Exam-Oriented Questions

- Derive ADC resolution formula
- Explain SAR ADC architecture
- Explain ADC initialization steps
- Why sampling time is important?

Lecture Summary

- ADC converts analog to digital
- STM32 uses 12-bit SAR ADC
- Polling-based ADC is simple
- Interrupt-based ADC improves efficiency

DMA Driver Development in STM32

Lecture Objectives

After this lecture, students will be able to:

- Understand Direct Memory Access concept
- Explain DMA architecture in STM32
- Configure DMA for peripheral-to-memory transfer
- Interface ADC with DMA
- Understand circular and normal modes

Why DMA is Required?

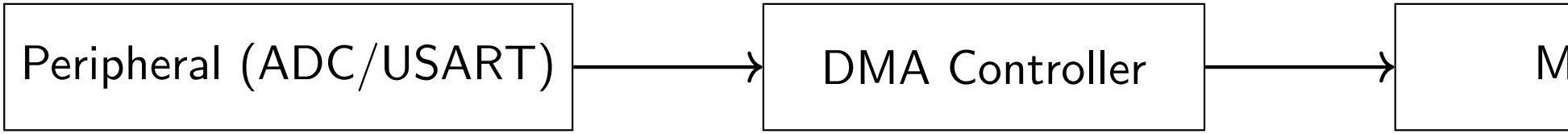
- CPU-based data transfer is inefficient
- High-speed peripherals require fast memory movement
- Polling or interrupt-based data movement increases CPU load

Core Idea

DMA transfers data without CPU intervention

DMA Concept

- Peripheral generates DMA request
- DMA controller transfers data
- CPU remains free



STM32 DMA Architecture

- DMA1 and DMA2 controllers
- Each controller has multiple streams
- Each stream supports multiple channels
- Supports:
  - Peripheral to Memory
  - Memory to Peripheral
  - Memory to Memory

Important DMA Registers

- DMA_SxCR – Stream configuration
- DMA_SxNDTR – Number of data
- DMA_SxPAR – Peripheral address
- DMA_SxM0AR – Memory address
- DMA_LISR / HISR – Status register

DMA Transfer Steps

1. Enable DMA clock
2. Configure stream
3. Set peripheral address
4. Set memory address
5. Set number of data
6. Enable stream

Enable DMA Clock

```
#define RCC_AHB1ENR (*(volatile unsigned int*)0x40023830)

void dma_clock_enable(void)
{
 RCC_AHB1ENR |= (1 << 21);  // Enable DMA1
}
```

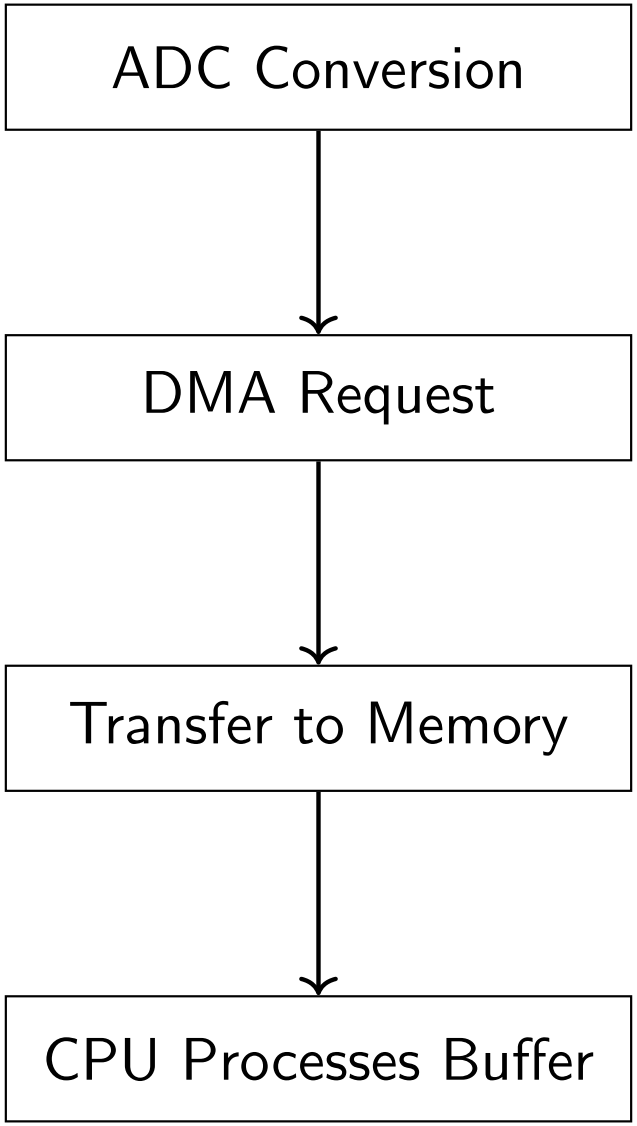
DMA Configuration Example (ADC to Memory)

```
#define DMA1_S0CR   (*(volatile unsigned int*)0x40026010)
#define DMA1_S0NDTR (*(volatile unsigned int*)0x40026014)
#define DMA1_S0PAR  (*(volatile unsigned int*)0x40026018)
#define DMA1_S0M0AR (*(volatile unsigned int*)0x4002601C)

void dma_config(unsigned int *buffer)
{
 DMA1_S0PAR  = 0x4001204C;  // ADC1_DR address
 DMA1_S0M0AR = (unsigned int)buffer;
 DMA1_S0NDTR = 10;          // Transfer 10 samples

 DMA1_S0CR |= (1 << 0);     // Enable stream
}
```

DMA Modes

- Normal Mode:
  - Stops after transfer complete
- Circular Mode:
  - Automatically restarts
  - Used for continuous ADC sampling

DMA + ADC Flow


ADC Conversion
DMA Request
Transfer to Memory
CPU Processes Buffer

Advantages of DMA

- High-speed data transfer
- Reduced CPU load
- Suitable for:
  - Audio processing
  - Sensor data logging
  - Communication buffers

DMA Interrupt

- Transfer complete interrupt
- Half-transfer interrupt
- Transfer error interrupt

Polling vs Interrupt vs DMA

Feature	Polling	Interrupt	DMA
CPU Load	High	Medium	Low
Speed	Low	Medium	High
Industrial Use	Rare	Common	Very Common

Common Mistakes

- Wrong stream selection
- Not disabling stream before reconfiguration
- Incorrect peripheral address
- Missing circular mode for continuous sampling

Exam-Oriented Questions

- Explain DMA working with diagram
- Differentiate normal and circular mode
- Why DMA preferred over interrupt?
- Explain ADC-DMA interfacing

Lecture Summary

- DMA enables high-speed data transfer
- Works independently of CPU
- Supports multiple transfer modes
- Essential for real-time embedded systems

I2C Protocol and Driver Development

Lecture Objectives

After this lecture, students will be able to:

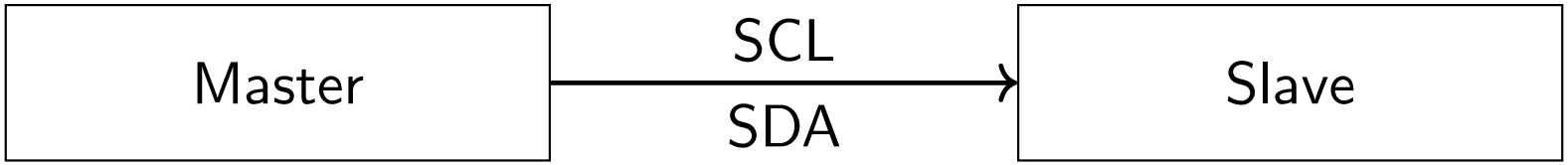
- Understand I2C protocol fundamentals
- Explain master-slave communication
- Understand START and STOP conditions
- Configure I2C in STM32
- Perform basic data transmission

Why I2C is Important?

- Two-wire communication protocol
- Supports multiple slaves
- Widely used for:
  - Sensors
  - EEPROM
  - RTC modules
  - Displays

I2C Signals

- SDA – Serial Data Line
- SCL – Serial Clock Line



Open-Drain Concept

- I2C lines use open-drain outputs
- External pull-up resistors required
- Devices only pull line LOW

I2C Communication Steps

1. START condition
2. Slave address transmission
3. ACK/NACK
4. Data transmission
5. STOP condition

START and STOP Conditions

- START: SDA goes LOW while SCL HIGH
- STOP: SDA goes HIGH while SCL HIGH

I2C Addressing

- 7-bit addressing (most common)
- 10-bit addressing (rare)
- Last bit indicates:
  - 0 – Write
  - 1 – Read

STM32 I2C Registers

- I2C_CR1 – Control register 1
- I2C_CR2 – Control register 2
- I2C_SR1 – Status register 1
- I2C_SR2 – Status register 2
- I2C_DR – Data register
- I2C_CCR – Clock control register

I2C Initialization Steps

1. Enable I2C clock
2. Configure GPIO pins in AF open-drain mode
3. Configure clock frequency
4. Enable I2C peripheral

Enable I2C Clock

```
#define RCC_APB1ENR (*(volatile unsigned int*)0x40023840)

void i2c1_clock_enable(void)
{
 RCC_APB1ENR |= (1 << 21);  // Enable I2C1
}
```

Enable I2C Peripheral

```
#define I2C1_CR1 (*(volatile unsigned int*)0x40005400)

void i2c1_enable(void)
{
 I2C1_CR1 |= (1 << 0);  // PE bit
}
```

Generating START Condition

- Set START bit in CR1
- Wait for SB flag in SR1

Generate START

```
void i2c_start(void)
{
 I2C1_CR1 |= (1 << 8);  // START bit
 while(!(I2C1_SR1 & (1 << 0)));  // Wait SB
}
```

Sending Slave Address

- Write address to DR
- Wait for ADDR flag

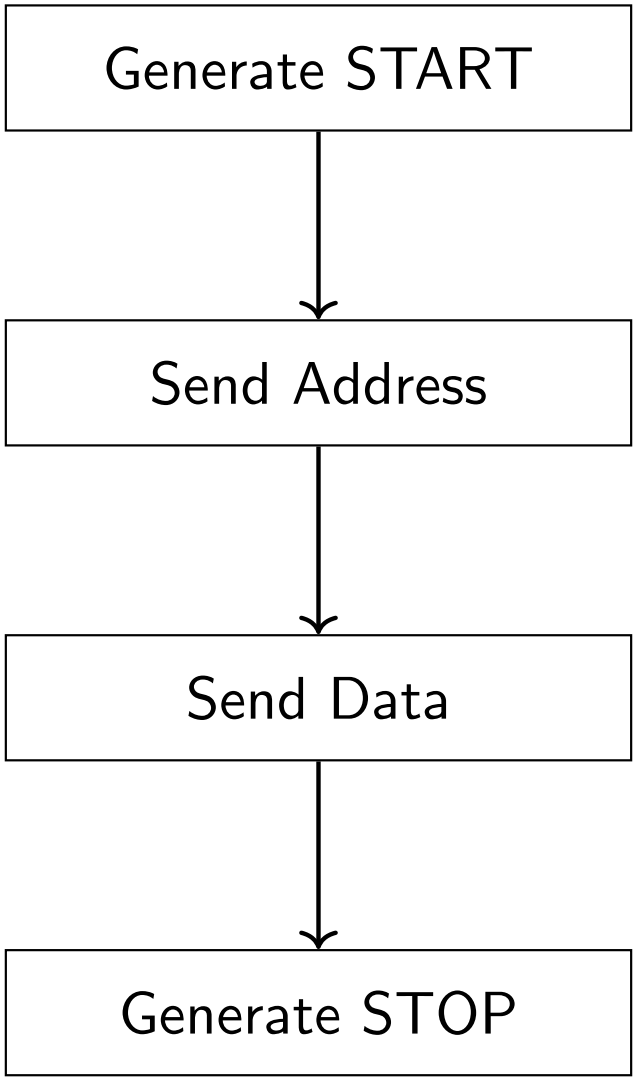
Send Address

```
#define I2C1_DR  (*(volatile unsigned int*)0x40005410)
#define I2C1_SR1 (*(volatile unsigned int*)0x40005414)

void i2c_send_address(unsigned char addr)
{
 I2C1_DR = addr;
 while(!(I2C1_SR1 & (1 << 1)));  // Wait ADDR
}
```

Sending Data

- Write data to DR
- Wait for TXE flag

I2C Communication Flow



Advantages of I2C

- Only two wires
- Multiple slaves supported
- Simple hardware

Limitations

- Slower than SPI
- Pull-up resistors required
- Bus arbitration complexity

Exam-Oriented Questions

- Explain I2C protocol with diagram
- What is open-drain configuration?
- Explain START and STOP conditions
- Compare I2C and SPI

Lecture Summary

- I2C is two-wire serial protocol
- Uses master-slave architecture
- Requires open-drain configuration
- STM32 supports full hardware I2C

SPI Protocol and Driver Development

Lecture Objectives

After this lecture, students will be able to:

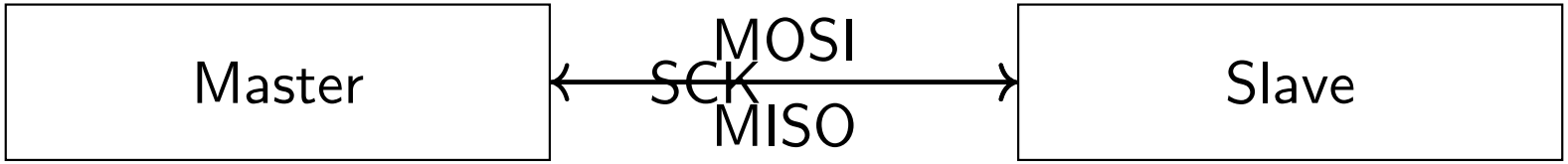
- Understand SPI protocol fundamentals
- Explain master-slave full duplex communication
- Configure SPI in STM32
- Perform SPI data transmission
- Compare SPI with I2C

Why SPI is Important?

- High-speed communication protocol
- Used for:
  - Displays
  - ADC/DAC chips
  - SD cards
  - Wireless modules
- Faster than I2C

SPI Signals

- MOSI – Master Out Slave In
- MISO – Master In Slave Out
- SCK – Serial Clock
- NSS – Slave Select



SPI Characteristics

- Full duplex communication
- No addressing
- Master controls clock
- Multiple slaves via NSS

SPI Clock Polarity and Phase

- CPOL – Clock Polarity
- CPHA – Clock Phase
- Four SPI modes:

$$Mode = (CPOL, CPHA)$$

Examples: Mode 0 = (0,0) Mode 3 = (1,1)

STM32 SPI Registers

- SPI_CR1 – Control Register 1
- SPI_CR2 – Control Register 2
- SPI_SR – Status Register
- SPI_DR – Data Register

SPI Initialization Steps

1. Enable SPI clock
2. Configure GPIO pins as AF
3. Set baud rate
4. Configure master mode
5. Enable SPI

Enable SPI Clock

```
#define RCC_APB2ENR (*(volatile unsigned int*)0x40023844)

void spi1_clock_enable(void)
{
 RCC_APB2ENR |= (1 << 12);  // Enable SPI1
}
```

SPI Configuration Example

```
#define SPI1_CR1 (*(volatile unsigned int*)0x40013000)

void spi1_init(void)
{
 SPI1_CR1 |= (1 << 2);   // Master mode
 SPI1_CR1 |= (3 << 3);   // Baud rate
 SPI1_CR1 |= (1 << 6);   // SPI enable
}
```

SPI Data Transmission

- Wait until TXE flag set
- Write data to DR
- Wait until RXNE flag set
- Read DR

SPI Transmit Function

```
#define SPI1_SR (*(volatile unsigned int*)0x40013008)
#define SPI1_DR (*(volatile unsigned int*)0x4001300C)

void spi1_send(unsigned char data)
{
 while(!(SPI1_SR & (1 << 1)));  // TXE
 SPI1_DR = data;
 while(!(SPI1_SR & (1 << 0)));  // RXNE
 unsigned char temp = SPI1_DR;
}
```

SPI vs I2C Comparison

Feature	SPI	I2C
Speed	High	Moderate
Wires	4	2
Addressing	No	Yes
Full Duplex	Yes	Half Duplex
Complexity	Simple	Moderate

Advantages of SPI

- High speed
- Full duplex
- Simple hardware

Limitations

- More wires required
- No built-in acknowledgment
- Separate NSS for each slave

Common Mistakes

- Wrong SPI mode
- Incorrect baud rate selection
- Forgetting to configure GPIO AF
- Not reading DR after transmission

Exam-Oriented Questions

- Explain SPI protocol with diagram
- What are SPI modes?
- Compare SPI and I2C
- Explain full duplex communication

Unit II Completion Summary

- ARM + STM32 architecture
- GPIO theory and coding
- USART polling and interrupt
- SysTick and General Timers
- ADC and DMA
- I2C and SPI communication

Outcome

Students can now develop complete embedded drivers

Unit II Completed Successfully